

Dossier de Conception - Version Finale

1. Contexte du Projet

Le projet s'inscrit dans le projet Optimod'Lyon et a pour objectif de fournir une application permettant de charger un plan et des demandes Pickup & Delivery afin de préparer une tournée.

JavaFX constitue la base de notre architecture IHM, nous l'utilisons pour le changement de scènes et l'interaction utilisateur.

Pour cette version finale, la fonctionnalité principale consiste à :

- charger le fichier XML du plan,
- afficher la carte
- charger et afficher les Pickup & Delivery
- afficher les points de Pickup & Delivery
- consulter son trajet/obtenir la tournée
- générer un pdf avec le nom de rue des tournées
- ajouter ou supprimer une livraison
- déplacer un Pickup et/ou une Delivery
- modifier l'ordre de priorité d'un Pickup et/ou d'une Delivery

2. Architecture Logicielle

2.1 Vue d'ensemble

L'application adopte une **architecture MVC (Modèle – Vue – Contrôleur)**.

Ce choix permet de séparer proprement l'interface utilisateur, la logique métier et les données, afin de garantir la lisibilité du code et de faciliter les futures évolutions fonctionnelles.

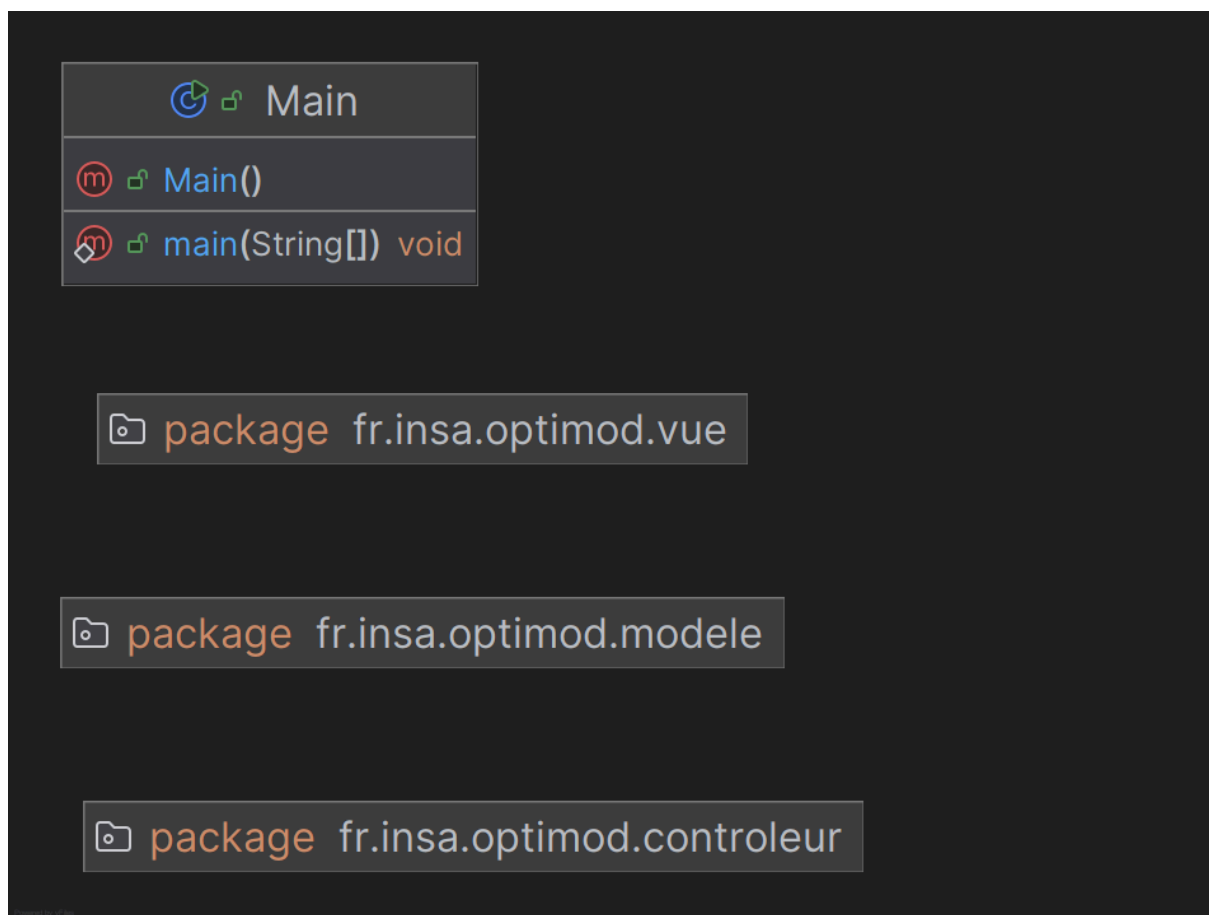
- **Modèle** : représente les données métiers manipulées par l'application (intersections, segments, graphe, requêtes Pickup&Delivery, tournée). C'est également dans cette couche que se trouvent les algorithmes de traitement (extraction du graphe utile, routage avec A*).
- **Vue** : regroupe l'ensemble des écrans et interfaces graphiques (page d'accueil,

page d'affichage de la carte, etc). La vue n'effectue aucun traitement, elle affiche uniquement les informations fournies par les contrôleurs.

- **Contrôleur** : coordonne les interactions entre l'utilisateur et le reste du système. Il reçoit les actions (chargement de fichiers, demande de génération de tournée), appelle les services du modèle, puis met à jour la vue en fonction des résultats.

L'architecture MVC mise en place en V1 s'appuie toujours sur JavaFX pour gérer la vue.

2.2 Diagramme de packages



2.3 Description des packages

fr.insa.optimod.modele:

Contient l'ensemble des classes représentant les **données métiers** du projet.

Ce package correspond au **Modèle** dans l'architecture MVC.

Il inclut les structures nécessaires pour représenter :

- la carte (nœuds + tronçons),
- les demandes (livraisons + entrepôt),
- les objets manipulés par le contrôleur.

fr.insa.optimod.vue:

Contient les classes responsables de l'**interface utilisateur**. Elles affichent les pages de l'application (chargement XML, bouton de génération, écran de résultat) et ne font aucun traitement. Ce package correspond à la **Vue (V)** du MVC.

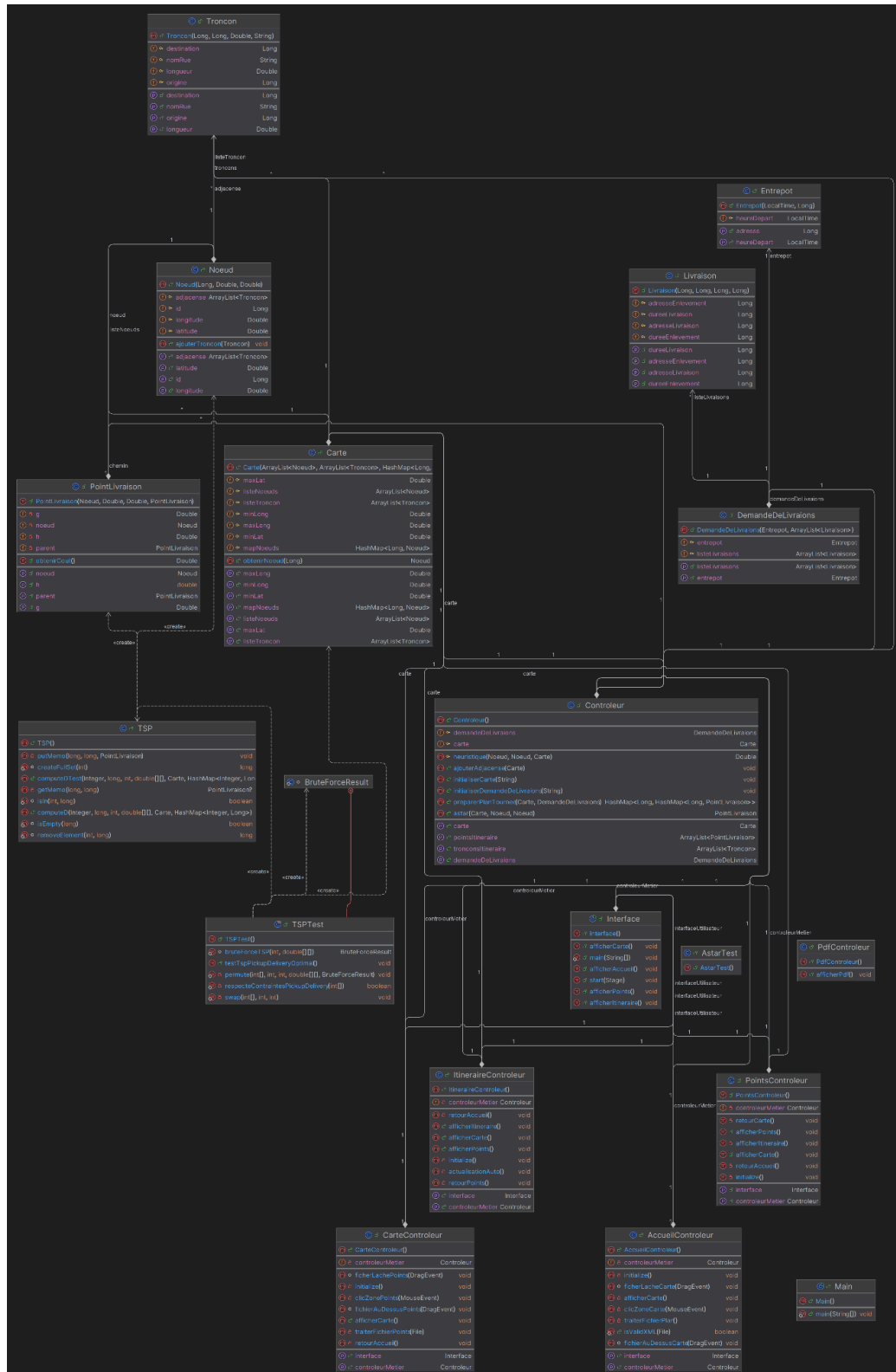
fr.insa.optimod.controleur:

Contient les classes qui assurent la **liaison entre la vue et le modèle**.

Les contrôleurs récupèrent les actions utilisateur (choix de fichier, clic sur bouton), appellent les méthodes du modèle, et transmettent les résultats à la vue.

C'est la partie **Contrôleur (C)** du MVC.

3.1 Diagramme de classes



3.2 Description des classes principales

Main

Point d'entrée de l'application.

Rôle : lancer JavaFX et afficher la fenêtre principale.

Carte

Représente le plan de la ville.

Attributs : liste de nœuds, liste de tronçons, bornes géographiques.

Rôle : stocker la structure du graphe routier chargé depuis le XML.

Noeud

Représente une intersection.

Attributs : id, latitude, longitude, tronçons sortants.

Rôle : élément de base du graphe utilisé par A*.

Troncon

Représente un segment routier.

Attributs : origine, destination, nom de rue, longueur.

Rôle : relier deux nœuds.

PointLivraison

Point de pickup ou de livraison.

Attributs : nœud associé, durée, type (pickup/delivery).

Rôle : unité manipulée par le TSP pour construire la tournée.

Livraison

Définit un couple pickup/livraison.

Attributs : adresses, durées d'enlèvement et de livraison.

Rôle : vérifier et respecter la contrainte Pickup → Delivery.

Entrepot

Point de départ du livreur.

Attributs : adresse, heure de départ.

Rôle : initialiser la tournée.

DemandeDeLivraisons

Regroupe les livraisons et l'entrepôt.

Attributs : liste de livraisons, entrepôt.

Rôle : structure d'entrée pour le calcul de tournée.

AStarTest

Classe de test du A*.

Rôle : vérifier la validité des distances et itinéraires générés.

TSP

Calcule la tournée optimale via programmation dynamique.

Attributs : matrice des coûts, contraintes Pickup → Delivery.

Rôle : déterminer l'ordre optimal de visite.

TSPTest

Classe de test du TSP.

Rôle : valider la cohérence des tournées calculées.

Interface

Gestionnaire central de l'IHM (JavaFX).

Rôle : charger les scènes, gérer la fenêtre et les transitions.

AccueilControleur

Contrôleur de la page d'accueil.

Rôle : importer la carte XML et passer à la vue suivante.

CarteControleur

Contrôleur de la scène d'affichage de la carte.

Rôle : afficher le plan, les tronçons et importer les livraisons.

PointsControleur

Gère l'affichage des pickups/livraisons.

Rôle : placer et styliser les points sur la carte.

ItineraireControleur

Gère l'affichage du trajet calculé.

Rôle : afficher le chemin optimal et les étapes.

PdfContrôleur

Génère le PDF récapitulatif.

Rôle : produire le document final de la tournée.

Contrôleur

Contrôleur métier principal.

Rôle : orchestrer l'ensemble du processus (import, calcul A*, calcul TSP, affichage).

Implémente l'algorithme A* (calculer les plus courts chemins entre nœuds) car la méthode est assez courte, nous avons donc décidé de la laisser dans le contrôleur.

4. Algorithmes

4.1 A* : calcul des plus courts chemins

Nous utilisons A* pour calculer les plus courts chemins entre les différents points utiles (entrepôt, pickups, deliveries).

Ce choix s'explique car :

- A* est plus rapide que Dijkstra grâce à son heuristique admissible (distance géographique).
- Il cible directement la destination au lieu d'explorer tout le graphe.
- Il permet d'obtenir efficacement la matrice des distances réelles nécessaire au calcul de la tournée.

A* sert donc à déterminer le coût minimal entre chaque paire de points.

4.2 Programmation dynamique : calcul de la tournée

Une fois les distances obtenues grâce à A*, nous utilisons une **programmation dynamique** pour déterminer l'ordre optimal de visite des points.

Cette méthode permet :

- de respecter les contraintes Pickup → Delivery,
- d'obtenir une tournée cohérente à partir des distances calculées,
- de garder un contrôle total sur l'algorithme.

Nous avons choisi d'implémenter nous-mêmes les algorithmes A* et TSP afin de maîtriser entièrement leur fonctionnement et d'assurer une meilleure flexibilité. Cette approche nous permet d'adapter facilement les algorithmes aux besoins spécifiques du projet, d'intégrer de nouvelles contraintes (comme le Pickup & Delivery pour le TSP) et de faire évoluer le système sans dépendre d'une bibliothèque externe difficile à modifier.

5.Choix des librairies

On utilise PDFBox pour créer le pdf car c'est une librairie open Source et c'est bien documenté. De même pour la réalisation des tests qui se sont fait avec la librairie de Tests JUnit qui est complètement open source.

6.Tests

Une librairie de test a été utilisée pour faciliter ces tests, la librairie Junit Jupiter.

6.1. Test TSP: Validité de l'algorithme du tsp (DP) avec contraintes

Objectif

Vérifier que notre implémentation du TSP avec enlèvements avant livraisons, basée sur l'algorithme Held-Karp (programmation dynamique), calcule bien :

- un coût optimal,
- un chemin valide,
- respectant les contraintes (« pickup avant livraison »).

Méthode

1. Construction d'une matrice de coûts cohérente entre 7 points.
2. Exécution de notre algorithme DP TSP (computeDTest).
3. Exécution d'un solveur brute-force générant toutes les permutations, filtrées par la contrainte P&D (fait avec l'aide d'une IA pour une meilleure rapidité).
4. Comparaison du meilleur coût brute-force vs DP.

Ce test est validé.

6.2. Test A* Cas simple (chemin direct optimal)

Objectif

Valider que l'algorithme A* retrouve correctement le chemin le plus court dans un petit graphe maîtrisé, issu d'un fichier XML simple.

Méthode

- XML contenant 3 nœuds : 100, 200, 300
- Tronçons :
 - 100 → 200 (50)
 - 200 → 300 (50)
 - 100 → 300 (200)

Chemin optimal attendu : 100 → 200 → 300 (coût = 100/4.16666666667)

Ce test est validé.

6.3. Test A* Cas complexe (chemin optimal long vs raccourcis coûteux)

Objectif

Évaluer A* dans une situation où plusieurs « raccourcis » existent, mais où le vrai chemin optimal nécessite plus d'étapes.

Méthode

XML contenant 5 nœuds : 100, 200, 300, 400, 500

Deux types de tronçons :

1. Chemin optimal long :
 - 100 → 200 (50)
 - 200 → 300 (50)
 - 300 → 400 (50)
 - 400 → 500 (50)
 - Total = 200
2. Faux raccourcis plus chers :
 - 100 → 300 (130)
 - 100 → 500 (400)
 - etc.

Chemin optimal attendu : 100 → 200 → 300 → 400 → 500 (200/4.16666667)

Ce test est validé

6.4. Test A* Cas déconnecté (aucun chemin possible)

Objectif

S'assurer que A* gère correctement les cas où aucune route n'existe entre la source et la destination.

Méthode

XML contenant deux composantes disjointes :

(Aucun tronçon entre le groupe 100/200 et le groupe 300/400).

Chemin recherché : 100 → 400

→ Aucun chemin possible.

Ce test est validé.

7. Choix spécifiques et explications

7.1. Construction de la matrice de coûts pour le TSP

Le contrôleur prépare ensuite un **graphe complet réduit** entre l'entrepôt, les adresses d'enlèvement et de livraison :

```
public      HashMap<Long,      HashMap<Long,      PointLivraison>>
preparerPlanTournee(Carte carte, DemandeDeLivraisons demande)
```

Les choix importants :

- On commence par construire la liste et les hashmap des adresses pertinentes :

- l'entrepôt
- chaque enlèvement
- chaque livraison
- Pour **chaque paire de points (i, j)**, on lance A* pour obtenir :
 - le plus court chemin entre les deux adresses
 - le coût associé, stocké dans `cout[i][j]`
- On ajoute ensuite les **durées de service** (enlèvement, livraison) directement dans les coûts de la matrice afin que le TSP prenne en compte :
 - temps de trajet
 - **temps passé sur place** (déchargement, chargement)

En parallèle, nous stockons le chemin détaillé pour chaque paire dans la structure :

```
HashMap<Long, HashMap<Long, PointLivraison>> tournée;
```

qui associe, pour chaque sommet d'arrivée, les chemins A* vers tous les sommets de départ.

Cela permet de **réutiliser les résultats de A*** :

- pour le calcul du TSP (via `cout[][]`)
- pour reconstruire ensuite l'itinéraire complet (liste de tronçons) sans recalculer les plus courts chemins.

7.2. Reconstruction de l'itinéraire complet (tronçons + temps)

Enfin, deux étapes sont réalisées pour rendre le résultat exploitable par l'IHM :

1. Reconstruction de la tournée TSP en termes de points

- On remonte la chaîne des `PointLivraison` renvoyée par `computed` pour obtenir `tspListe` (ordre de visite : entrepôt, enlèvements, livraisons, retour entrepôt).
- On recalcule les coûts cumulés `g` pour chaque point en utilisant la matrice `cout[][]`, ce qui permet d'avoir un horaire/temps global cohérent.

2. Reconstruction des tronçons détaillés entre chaque point TSP

(`affichageTournée()`)

- Pour chaque paire successive (`courant`, `prochain`) de `tspListe`, on récupère dans `tournée` le chemin A* correspondant.
- On remonte les parents de ce `PointLivraison` pour reconstituer la séquence de tronçons, que l'on ajoute dans la liste `troncons`.

- On obtient ainsi une liste de **tronçons ordonnés** qui représente le trajet complet réel sur la carte, et une liste de **points de livraison** avec leur temps d'arrivée.
- A noter qu'on récupère aussi les nœuds traverser pour le pdf pour choisir les directions plus facilement.

Ce découplage entre :

- le TSP (qui travaille sur une matrice de coûts abstraite)
- et la reconstruction détaillée via les résultats A* mémorisés

permet d'avoir un code plus clair, mieux factorisé, et facile à faire évoluer (changement de coût, de vitesse, ajout de contraintes, etc.).

7.3. Choix des directions dans le pdf

Pour déterminer si le véhicule doit aller à gauche, à droite, tout droit ou effectuer un demi-tour, nous analysons l'orientation entre deux vecteurs de déplacement successifs :

v_1 : le déplacement précédent (de n_1 vers n_2)

v_2 : le déplacement suivant (de n_2 vers n_3)

1. Détection droite/gauche avec le produit vectoriel (2D)

Nous commençons par calculer le produit vectoriel entre v_1 et v_2 (dir). Ce terme indique le sens de rotation :

$\text{dir} > 0 \rightarrow$ rotation anti-horaire $\rightarrow$ tourner à gauche

$\text{dir} < 0 \rightarrow$ rotation horaire $\rightarrow$ tourner à droite

Pour éviter que de très faibles variations numériques soient interprétées comme un virage, nous utilisons une petite tolérance faite par expérience. Si $|\text{dir}| < 2 \cdot 10^{-7}$, nous considérons que les deux vecteurs sont quasi alignés.

2. Détection tout droit / demi-tour avec le produit scalaire

Lorsque les deux segments sont presque alignés, le produit vectoriel ne suffit plus. Nous utilisons alors le produit scalaire (sens). Il permet de distinguer :

$\text{sens} > 0 \rightarrow$ vecteurs orientés dans la même direction $\rightarrow$ tout droit

$\text{sens} < 0 \rightarrow$ vecteurs opposés $\rightarrow$ demi-tour